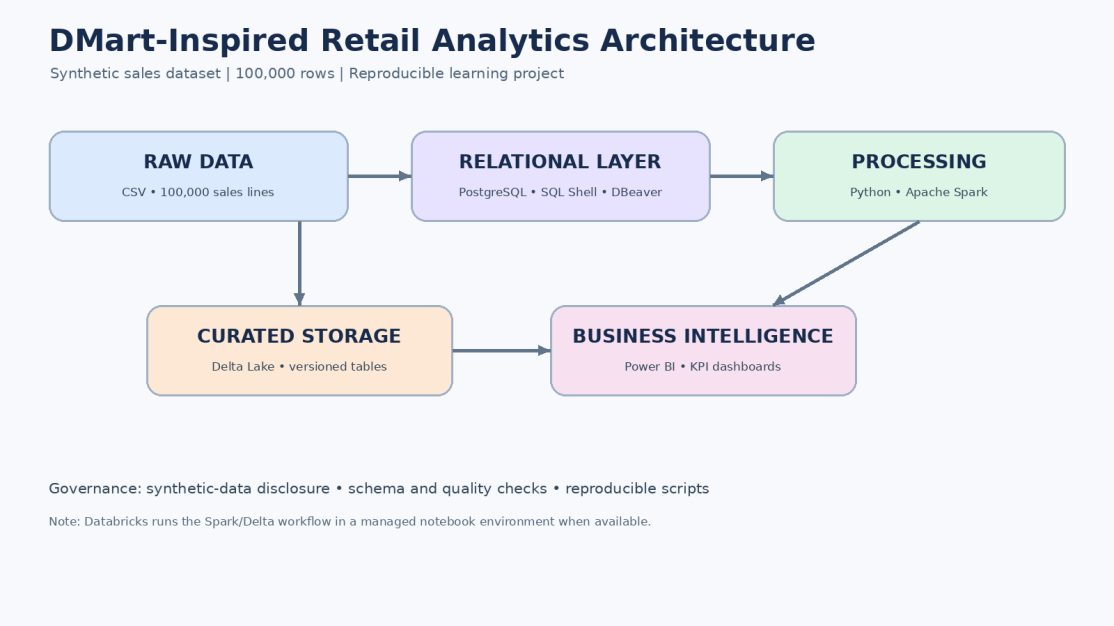
RETAIL SALES AND BUSINESS INTELLIGENCE

USING BIG DATA TECHNOLOGIES

A DMart-Inspired Case Study

PostgreSQL | DBeaver | Python | Apache Spark | Databricks | Delta Lake | Power BI



Prepared by: [Student Name]

[Programme / Institution]

Project Guide: [Faculty Name]

Submission Date: [Date]

Synthetic dataset - 100,000 sales lines - for education only

DECLARATION

I declare that this project report, titled “Retail Sales and Business Intelligence Using Big Data Technologies: A DMart-Inspired Case Study,” is prepared for academic learning and demonstration. The analysis pipeline and supporting files are designed to demonstrate data ingestion, database management, validation, transformation, and visualization using commonly available tools.

The dataset included with the project is synthetic. It was generated for educational use and is not collected from DMart or any confidential customer or point-of-sale system. Any company context is limited to publicly available information and is separated from the simulated transactions used for the technical exercises.

The findings reported from the generated dataset describe that dataset only. They must not be interpreted as evidence of DMart’s actual sales, customers, regional performance, product demand, discounts, or payment behaviour. Any code, outputs, and conclusions should be checked again after executing the included scripts in the student’s own environment.

Student signature: _____ *Date:* _____

CERTIFICATE (TEMPLATE)

This is to certify that the project report entitled “Retail Sales and Business Intelligence Using Big Data Technologies: A DMart-Inspired Case Study” is submitted by [Student Name], bearing [Student ID / Roll Number], in partial fulfilment of the requirements of [Programme / Course Name].

The project demonstrates a learning workflow using PostgreSQL, DBeaver, Python, Apache Spark, Databricks, Delta Lake, and Power BI. The implementation uses a synthetic retail dataset and is intended to illustrate methods of data management and analytics rather than report actual DMart operational results.

The final acceptance of this report is subject to the student completing the database, processing, and dashboard exercises, validating the outputs, and updating this report with the execution evidence required by the institution.

Project Guide: _____ *Head of Department:* _____ *Date:* _____

ACKNOWLEDGEMENT

I express my sincere gratitude to the faculty members and project guide for their guidance in defining the scope of this retail analytics project. Their emphasis on structured problem-solving, data quality, and clear communication helped shape the project as a practical exercise rather than a tool-only demonstration.

I also acknowledge the open-source communities and documentation maintainers behind PostgreSQL, Apache Spark, and Delta Lake, along with the creators of Python libraries used in data analysis. Their documentation makes it possible for students to learn core data engineering concepts using accessible software.

Microsoft Power BI and DBeaver provide useful interfaces for exploring data and presenting results. Public investor information published by DMart is used only as a source of company context; the transaction data in this project is synthetic and independent of the company.

Finally, I acknowledge the contribution of the project dataset and testing scripts as learning resources. The final report should be updated with the student’s actual screenshots, tool versions, run logs, and observations before academic submission.

ABSTRACT

Retail operations generate structured records that can be analyzed to understand sales patterns, product-category contribution, payment preferences, discount distribution, and changes over time. This project develops a reproducible demonstration pipeline for retail sales analytics using 100,000 synthetic sales-line records created for a DMart-inspired case study.

The proposed workflow starts with CSV ingestion into PostgreSQL, database exploration through SQL Shell and DBeaver, and data validation using SQL and Python. Apache Spark is used to demonstrate DataFrame transformations and grouped analytics. A Databricks notebook shows how the processing workflow can be hosted in a managed notebook environment, while Delta Lake demonstrates transactional storage and table-version history. Power BI is used to present key performance indicators in a management-oriented dashboard.

A preliminary validation of the included file found 100,000 rows, 15 columns, 0 missing values, and 0 duplicate sales-line IDs. Its calculated sum of Total_Amount is INR 31,706,049.38, and its mean amount per sales line is INR 317.06. These results describe the synthetic data only.

The project’s contribution is a transparent learning architecture, testable scripts, a data dictionary, and a clear distinction between technical demonstration and real-world business evidence.

TABLE OF CONTENTS

Front matter 1–7

Chapter 1: Introduction 8–9

Chapter 2: Retail context and conceptual background 10–11

Chapter 3: Objectives, scope, and questions 12

Chapter 4: Technology stack and architecture 13–14

Chapter 5: Dataset design and data model 15–17

Chapter 6: Methodology and implementation 18–25

Chapter 7: Results and validation 26–27

Chapter 8: Limitations and recommendations 28

Conclusion and future scope 29

References and appendix 30

Page references may need to be refreshed if the report is re-formatted. This document uses manual page breaks to target a 30-page layout. Replace the student/institution placeholders and insert genuine execution screenshots before submission.

LIST OF TABLES, FIGURES, AND ABBREVIATIONS

Planned tables: project objectives; software roles; dataset fields; quality controls; sample SQL questions; validation results; measured dataset summaries.

Planned figures: end-to-end architecture; CSV-to-database flow; raw/cleaned/aggregate layer model; Power BI executive dashboard; product-performance dashboard; data-quality check output.

Abbreviations used in this report:

DBMS — Database Management System; ETL — Extract, Transform, Load; KPI — Key Performance Indicator; CSV — Comma-Separated Values; SQL — Structured Query Language; BI — Business Intelligence; OLAP — Online Analytical Processing; SKU — Stock Keeping Unit; ACID — Atomicity, Consistency, Isolation, Durability; DAX — Data Analysis Expressions; EDA — Exploratory Data Analysis.

Screenshots and result tables should be sourced from actual runs in PostgreSQL, Spark, Databricks, or Power BI. Do not insert fabricated screenshots or treat conceptual architecture diagrams as execution evidence.

CHAPTER 1: INTRODUCTION

Retail organizations rely on consistent information to plan assortment, monitor sales, review discount activity, and communicate business performance. Individual transaction records are useful only when they can be loaded, validated, queried, and summarized in a reliable way. This project focuses on that data-to-insight sequence.

DMart is used as a recognizable public retail case-study context. Public company disclosures can help readers understand the business domain and corporate context, but they do not provide the transaction-level data used here. The technical analysis therefore uses synthetic records designed to resemble common retail sales-line structures.

The project connects the responsibilities of a database system with a modern analytics workflow. PostgreSQL provides relational storage and SQL querying. DBeaver offers a graphical interface. Python supports validation and single-machine exploratory analysis. Apache Spark illustrates structured processing, while Databricks and Delta Lake provide a notebook and lakehouse-oriented extension. Power BI presents the results for a business audience.

The intended outcome is not an unsupported claim about actual DMart performance. It is a documented and reproducible demonstration that can be rerun, checked, and extended by a student.

PROBLEM STATEMENT AND MOTIVATION

A retail analytics task often begins with data stored in flat files and ends with a report used by non-technical stakeholders. Between those points, analysts must verify column meanings, ensure amounts are calculated consistently, handle invalid records, structure data for querying, and build visuals that do not overstate what the data supports.

The learning problem addressed here is how to design a connected workflow that makes those stages explicit. A one-tool solution would show only a portion of the process: a SQL query alone does not demonstrate the pipeline, while a dashboard alone does not prove that input records were validated. Combining tools demonstrates the links between storage, transformation, quality control, and reporting.

A related challenge is responsible interpretation. Synthetic data is useful for skill-building but can create false confidence if it is presented as actual company data. The report therefore labels all generated records and calculated results, documents known limitations, and avoids claims about DMart's real stores, products, customers, or business outcomes.

The motivation is to build a project with reproducible code and visible checks instead of relying exclusively on screenshots or a static dashboard. The project is suitable for a student portfolio because each stage can be demonstrated independently and then connected into an end-to-end story.

CHAPTER 2: RETAIL CONTEXT AND CONCEPTUAL BACKGROUND

DMart is a retail brand operated by Avenue Supermarts Limited. The company's official investor-relations page publishes financial results and other investor information. That site is the preferred starting point for company context and official disclosures [1]. This project does not infer transaction-level facts from corporate financial results.

Retail analytics commonly organizes questions around product, date, store, price, quantity, discount, and payment method. These attributes support descriptive questions such as which category contributes the most sales in a given sample, whether monthly totals vary, and how sales-line counts differ by payment type.

The dataset has intentionally simulated city and store identifiers, product categories, customer identifiers, and prices. It should not be used to estimate market share, store coverage, current pricing, or consumer behaviour in India. Those questions require valid external evidence or authorized company data.

The conceptual link between retail context and data engineering is straightforward: business questions determine which fields and measures are required; data-model choices determine whether those measures can be calculated correctly; and dashboard design determines whether findings can be understood without confusing sales lines with invoices.

LITERATURE AND TECHNICAL CONCEPTS

A relational DBMS stores structured records in tables and supports querying through a defined schema. PostgreSQL documents a `COPY` command for moving data between tables and files, while its client-side `\copy` command is useful for student workflows where the CSV is on the local machine [2]. The repository uses a staging table so raw data can be inspected before being exposed to analytical tables.

Apache Spark SQL provides structured processing through SQL and DataFrame APIs. Its documentation explains how Spark can query structured data and connect to a variety of sources [3]. In this project Spark is used for typed transformations and grouped summaries, not because 100,000 rows require a distributed engine.

Delta Lake is an open-source storage layer that adds transactional guarantees, schema enforcement, and version history to data lake workflows [4]. These features allow a learning exercise to show how curated data can be reproduced or inspected after updates.

Power BI documentation describes the process of connecting to data, building reports, and sharing insights [5]. The project follows a similar separation: validate the source and measures first, then build visuals and labels that accurately communicate sales-line metrics.

CHAPTER 3: OBJECTIVES, QUESTIONS, AND SCOPE

The primary objective is to design and document an end-to-end retail analytics workflow using the stated toolset. Supporting objectives are to (1) load 100,000 records into PostgreSQL, (2) define clear data types and relationships, (3) verify quality constraints and calculated totals, (4) implement descriptive analysis in SQL and Python, (5) process the data with Spark, (6) demonstrate Delta Lake storage, and (7) translate the findings into a Power BI dashboard.

The principal questions are: What is the total sales amount in the synthetic dataset? Which categories contribute most to that sum? How do monthly totals change? How are sales lines distributed by city and payment method? How can data quality be tested consistently across SQL and Python?

The scope covers descriptive analytics and a learning-oriented batch pipeline. It does not include forecasting, inventory optimization based on real stock movements, profitability analysis, customer segmentation using personal profiles, or basket affinity. Those require additional fields and a clearly defined business objective.

The unit of analysis is one synthetic sales line. Because each row has a unique Transaction_ID and no shared Invoice_ID, average bill value and multi-product basket analysis are explicitly outside the current dataset’s supported scope.

CHAPTER 4: TECHNOLOGY STACK

PostgreSQL is the relational storage layer. The project defines staging and dimensional tables, uses constraints to restrict invalid values, and runs analytical SQL. SQL Shell (`psql`) supports command-line operations; PostgreSQL also allows loading data through client-side `\copy` where appropriate [2].

DBeaver is a database client for visually exploring tables, executing SQL, and checking result sets. It is optional: the database can also be used entirely through `psql`. Python and pandas provide convenient record-level validation, data profiling, and chart generation. NumPy supports numerical operations, while matplotlib creates report-ready charts.

Apache Spark is included to demonstrate DataFrame transformations and grouped aggregation. Databricks supplies a managed notebook environment when an account and appropriate compute are available. Delta Lake adds a versioned storage layer compatible with Spark [3, 4].

Power BI is the business intelligence layer, where DAX measures and visuals express the validated metrics. A dashboard should not be treated as its own data-quality check; its headline totals must be reconciled with SQL and Python calculations.

The tools have different roles. Repetition across tools is intentional: matching totals across independent execution paths helps reveal import, type, formula, or filtering mistakes.

Tool	Role in this project
PostgreSQL	Staging, dimensions, fact table, SQL aggregations
DBeaver / psql	Database access, loading, query review
Python / pandas	Validation, profiling, chart generation
Apache Spark	Typed DataFrame transformations and aggregation
Databricks	Managed notebook execution when available
Delta Lake	Reliable curated storage and version history
Power BI	KPI cards, trend charts, category analysis

SYSTEM ARCHITECTURE AND DATA FLOW

The logical architecture contains source, staging, validation, transformation, curated storage, and reporting layers. The source is the synthetic CSV. The staging table retains the source columns. Quality checks inspect nulls, key uniqueness, valid quantities and prices, discount range, and total-amount calculations.

The relational pathway loads the CSV to PostgreSQL and creates dimension tables for products, stores, and customers plus a sales fact table. This separates descriptive attributes from the measures used in aggregations. DBeaver can visualize these structures, while SQL queries answer the business questions.

The processing pathway reads the same source file using Python or Spark. The cleaned dataset is saved in a consistent format, and Spark aggregates sales by category and month. In a configured environment, Delta Lake can persist the curated tables and maintain operation history. The Databricks notebook packages the workflow into repeatable cells.

Power BI sits at the consumption end. It should connect either to the source CSV for a simple local model or to the curated database output for a more integrated architecture. The included architecture diagram summarizes these relationships; it is a proposed design, not evidence that every stage has already executed.

Layer	Contents	Key check
Source	Synthetic CSV	Expected columns and row count
Staging	Raw PostgreSQL table	No import errors
Silver	Typed and validated records	Amounts and keys pass checks
Gold	Category/month aggregates	Totals reconcile to Silver
Reporting	Power BI model	Headline totals match SQL

Figure 1. Refer to assets/architecture.png for the pipeline diagram.

CHAPTER 5: DATASET DESCRIPTION

The dataset contains 100,000 records and 15 columns, with dates spanning 2025-01-01 through 2025-12-31. It is approximately 12.64 MB as CSV. It includes sales-line keys, date and time, store and city, customer and product identifiers, product description, category and brand, quantity, unit price, discount percentage, payment method, and total amount.

The data has been generated specifically for software practice. The simulated fields make it possible to practice joins and grouped analysis, but they are not verified against DMart's actual catalogue or branch list. Some product names are variants of base product names and are assigned distinct product IDs; this is acceptable for a synthetic exercise but should be documented when interpreting product rankings.

The Total_Amount field is calculated as quantity multiplied by unit price and reduced by the discount percentage, rounded to two decimal places. The currency convention is Indian rupees (INR). All values should be validated on load rather than assumed correct.

One hundred thousand rows provide enough volume for a student to run multiple queries and test import workflows comfortably. It is not, by itself, a large-data benchmark. Spark and Delta Lake are included to learn portable patterns that can later be applied to much larger files.

Profile item	Current dataset
Rows / columns	100,000 / 15
Date range	2025-01-01 to 2025-12-31
Product / store IDs	500 / 10
Customer IDs	19,872
CSV size	12.64 MB

DATA DICTIONARY AND MEASURES

The 15 columns are grouped into identifiers (Transaction_ID, Store_ID, Customer_ID, Product_ID), time attributes (Transaction_Date, Transaction_Time), descriptive attributes (City, Product_Name, Product_Category, Brand, Payment_Method), numeric inputs (Quantity, Unit_Price, Discount_Percent), and the output measure (Total_Amount). Detailed definitions and suggested data types are provided in `docs/data_dictionary.md`.

The primary additive business measure is Total_Amount. Its sum represents the aggregate value of all generated sales lines after discount. Quantity can be summed to calculate units sold, while row count measures sales lines. An average of Total_Amount is the average value per line, not the average bill value.

Discount_Percent is a rate and should not be summed. A simple average can be used descriptively, but a weighted discount or list-price comparison would require an explicitly defined methodology and consistent base price. Unit_Price is an illustrative unit price, and no cost-of-goods or margin field exists; profitability therefore cannot be derived.

Date and time fields should be parsed as date/time values, not retained as arbitrary text for analytics. IDs should be handled as text because they are identifiers rather than quantities. Currency and percentage formatting belongs in the reporting layer.

DATABASE DESIGN AND RELATIONAL MODEL

The database design uses a staging table that mirrors the CSV. Its purpose is to make the import auditable and preserve the raw field names. A separate fact table stores the foreign keys and measures. Dimension tables store product descriptions, store/city labels, and customer identifiers. This is a simple star-schema pattern suitable for reporting.

`Transaction_ID` is the primary key for the present dataset because the generated file has one unique identifier per row. The `fact_sales` table links to `dim_product`, `dim_store`, and `dim_customer`. Date can be grouped directly from the fact table for this starter design; an optional date dimension can be added later to support fiscal calendars and consistent date intelligence.

Database constraints check positive quantity and unit price, a discount rate between zero and one hundred, nonnegative sales amount, and unique keys. Indexes on date and common join keys support queries. At 100,000 records, the project should not assume indexing alone will lead to a meaningful performance improvement; timings depend on hardware, cache state, query plan, and indexes.

If a future version introduces one invoice ID shared across multiple product lines, the grain and measures should be revisited. That change would enable a more accurate average invoice value and basket analysis.

Table	Purpose	Key
staging_retail_sales	Raw imported file	transaction_id
dim_product	Product descriptors	product_id
dim_store	Simulated store labels	store_id
dim_customer	Synthetic customer keys	customer_id
fact_sales	Sales measures and foreign keys	transaction_id

CHAPTER 6: METHODOLOGY

The project follows a sequence of requirements definition, data inspection, loading, validation, normalization, analysis, transformation, storage, visualization, and reporting. Each stage leaves an artifact that can be checked or rerun. The included scripts use relative paths from the repository root where practical so the project can be moved between computers.

First, inspect the CSV header, row count, data types, missing values, distinct IDs, and numeric ranges. Second, create the database objects and load the raw data. Third, populate the dimension and fact tables from staging. Fourth, run SQL quality checks and compare calculated totals with the stored Total_Amount.

Fifth, run Python validation and exploratory summaries; the scripts generate CSV result tables and charts in an outputs directory. Sixth, run the Spark pipeline in a compatible local or managed environment. Seventh, use the Databricks notebook and Delta Lake operations where the chosen account supports them. Finally, build the dashboard and reconcile Power BI totals with SQL output.

This project is primarily descriptive. It uses defined checks and transparent aggregations rather than inferential statistics, prediction, or causal analysis. No business intervention is tested, so the results do not establish that discounts or payment methods cause a change in sales.

Phase	Action	Evidence
1. Inspect	Read schema and file profile	Column/row count
2. Load	Import CSV into staging	Load confirmation
3. Validate	Run SQL and Python checks	Zero unexpected errors
4. Transform	Create dims/fact and Spark layers	Counts and aggregates
5. Report	Build Power BI visuals	Reconciled dashboard total

POSTGRESQL IMPLEMENTATION

The repository contains scripts for database creation, table definition, CSV import, data quality checks, and business analysis queries. Run the database-creation script while connected to a maintenance database, then connect to `dmart_analytics` before executing the table script. Database creation is a separate step because PostgreSQL does not allow CREATE DATABASE inside a transaction block.

The import script uses the psql `\copy` meta-command to read the CSV from the client's working directory. When running from the repository root, the relative path points to `data/dmart_retail_sales_synthetic_100000.csv`. If using DBeaver, its Import Data wizard is a practical alternative; the psql-specific command may not execute in a normal SQL editor.

After import, populate the dimension tables from distinct source keys and then populate the fact table. Run the quality-check script to confirm counts, key uniqueness, ranges, and amount calculations. If loading the same data more than once, use the provided conflict handling or truncate staging and reload intentionally to avoid stale records.

The table design is deliberately explicit so students can see each type, key, and constraint. In an enterprise system, permissions, schema ownership, audit fields, incremental loads, and secret management would need further design.

Step	Command / action
Create database	psql -f database/01_create_database.sql
Create tables	psql -d dmart_analytics -f database/02_create_tables.sql
Load CSV	psql -d dmart_analytics -f database/03_import_csv.sql
Check quality	Run database/04_quality_checks.sql
Run queries	Run database/05_business_analysis_queries.sql

SQL ANALYSIS AND BUSINESS QUESTIONS

The business-analysis script starts with total sales amount, units sold, sales-line count, and average sales-line value. It then groups sales by product category, month, city, payment method, discount band, and product. These queries illustrate core SQL operations: aggregation, grouping, date truncation, CASE expressions, joins, ordering, and limiting results.

The category query uses the product dimension to access category labels and sums the fact-table amount. The monthly query groups transaction dates by calendar month. The city query joins stores to sales. The payment query can show both count and amount because these fields measure different outcomes: a payment method with more sales lines need not have the highest total value.

SQL outputs should be saved or captured after execution. For reproducibility, record the database version, dataset file hash if used, execution date, row counts, and query names. Avoid copying results from an earlier run if the dataset changes.

The included queries do not claim causal relationships. A higher sales amount in a category is a descriptive finding for this sample and may be influenced by the synthetic price and product-selection logic. It does not demonstrate actual demand, store strategy, or margin.

Question	SQL technique	Output
How much sales value?	SUM / AVG / COUNT	Overall KPI row
Which categories lead?	GROUP BY + JOIN	Category ranking
How do months compare?	DATE_TRUNC	Monthly trend
How is payment used?	GROUP BY method	Count and sales value
Which products lead?	ORDER BY + LIMIT	Top-ten product list

PYTHON VALIDATION AND EXPLORATORY ANALYSIS

The Python validator reads the CSV, checks required columns, detects missing values and duplicate sales-line IDs, verifies quantity and price constraints, checks the discount range, and recomputes the expected total. It exits with an error if validation fails. This is preferable to relying on manual visual inspection of a few rows.

The EDA script creates a compact KPI table and grouped summaries for product category, month, and city. It also saves charts for category sales and monthly sales. These files are generated in `outputs/` after running the script; they are not committed as precomputed outputs because regenerating them is part of the learning workflow.

Python is useful here because it supports quick profiling and chart generation with relatively little code. The purpose is not to duplicate every SQL result for its own sake, but to validate that different tools agree on critical metrics. For example, the grand total from pandas should match the PostgreSQL total within two decimal places.

EDA must be interpreted with care. The synthetic generator determines the distribution of stores, product categories, quantities, discounts, and payments. A chart reveals properties of that generator as much as it reveals retail patterns.

Output file	Content
kpis.csv	Counts, units, total and average line value
category_summary.csv	Category sales, units and line count
monthly_summary.csv	Month-level sales and line counts
city_summary.csv	Simulated city sales and line counts
sales_by_category.png	Category sales chart
monthly_sales.png	Month-level trend chart

APACHE SPARK PROCESSING

The Spark script reads the CSV using schema inference and then explicitly casts important columns to appropriate types. It checks for invalid quantities, prices, discounts, and missing transaction IDs, removes duplicate sales-line keys, derives a month field, and creates category and monthly aggregations. Output is written in Parquet by default to avoid requiring Delta configuration for the first local run.

Spark is designed to express structured transformations through DataFrames and SQL. Its optimizer can plan operations across partitions and its APIs provide a consistent way to query structured data [3]. The same conceptual steps used by pandas—read, validate, derive, group, write—can therefore be transferred to a distributed processing environment.

For this 100,000-row dataset, Spark may be slower than pandas because it has startup and execution overhead. That is not a failure: the point is to learn how the pipeline is written and how it could scale, then measure performance fairly with larger workloads if desired.

For a meaningful benchmark, use the same input, same output logic, repeated runs, and controlled conditions. Document machine specifications, Spark version, memory configuration, cache state, and run time. Do not report a speed advantage unless it was measured.

Spark operation	Purpose
Explicit casts	Use date, integer, decimal types
Validation filter	Reject invalid quantities, prices, discounts, or keys
dropDuplicates	Remove repeated sales-line identifiers
Derived columns	Calculated_Total and Sales_Month
Grouped writes	Save category/month aggregates as Parquet by default

DATABRICKS NOTEBOOK WORKFLOW

The included notebook organizes the workflow into configuration, raw ingestion, Bronze persistence, Silver validation, Gold aggregation, and optional Delta history inspection. Before running it, upload the CSV into an approved workspace volume or storage location and replace the placeholder path. The notebook is not expected to run unchanged because workspace catalogs, storage paths, permissions, and compute options differ by account.

The Bronze layer retains a source-like snapshot. Silver applies type conversion, key deduplication, and calculation checks. Gold produces category and monthly aggregates intended for downstream reporting. These layers are a useful naming convention and make it easier to explain which level of transformation has been applied.

Managed compute can require account setup, quotas, or usage credits; availability varies. The project can still be completed in local PostgreSQL and Python if Databricks access is not available, while the notebook remains part of the repository as a cloud-execution template.

Notebook outputs should only be captured after actually executing the notebook. When submitting screenshots, ensure that the displayed counts and totals correspond to the included CSV and that no placeholder path or error remains visible.

Layer	Typical contents	Use
Bronze	Raw imported records	Source snapshot
Silver	Typed, deduplicated records	Quality-controlled analysis
Gold	Category/month summaries	Business reporting
Notebook outputs	Counts, quality metrics, tables	Execution evidence

DELTA LAKE OPERATIONS

Delta Lake is an open-source table format and storage layer built to support reliable lakehouse workflows. Its documentation describes transaction support, schema enforcement, version history, and compatibility with Spark APIs [4]. In this project it is used to persist validated sales rows and inspect table history after writes.

The provided operations script reads the data, applies a limited type conversion and key-deduplication step, writes a Delta table, prints operation history, and demonstrates how a later version can be inspected. Delta support must be installed and configured to match the Spark runtime. A plain PySpark installation may not include the necessary Delta extensions.

Time travel should be demonstrated only after verifying that the expected versions are present. Retention and cleanup behaviour depend on configuration; a demonstration should not run destructive cleanup commands on data that is still needed. Delta versioning is not a substitute for a broader backup or retention strategy.

For a dataset this small, Delta Lake's primary value is instructional: it helps explain reproducible tables and controlled updates. If the project is expanded later, the same pipeline can be used to study schema evolution, incremental ingestion, and slowly changing dimensions.

Delta capability	Learning demonstration
Transactional table writes	Write a curated sales table
History	Inspect commits after writes
Time travel	Read a prior version when available
Schema checks	Prevent accidental schema drift with an explicit schema
Caution	Match Delta package version to Spark runtime

POWER BI DASHBOARD DESIGN

The Power BI layer translates verified aggregates into a small set of visuals. The recommended executive-summary page contains KPI cards for total sales amount, units sold, sales-line count, and average sales-line value. A monthly trend line and a category bar chart provide time and product context. Slicers allow the reader to filter date, city, category, and payment method.

A product page can show the top ten products by sales amount, units by category, and average discount. A store/payment page can compare simulated cities and payment methods. Use clear labels, currency formatting in INR, and tooltips that include both the metric and the applied filters.

DAX measures are provided in `powerbi/DAX_Measures.dax`; the theme file defines a consistent palette. Rename the imported table to `RetailSales` so the measures match. If the model uses PostgreSQL dimension and fact tables instead, adjust the DAX references and create relationships before building visuals.

The dashboard must state that data is synthetic. Avoid the phrase “average bill value” because no shared invoice key exists. After building the report in Power BI Desktop, save the `.pbix` file and insert screenshots into the report; the repository does not claim a Power BI binary was prebuilt.

Visual	Question answered
KPI cards	What is total sales and units sold?
Line chart	How does sales change by month?
Category bar chart	Which category contributes most?
Product table	Which synthetic SKUs rank highest?
Payment chart	How do payment method counts differ?
Slicers	How do filters change the results?

CHAPTER 7: PRELIMINARY RESULTS

The included synthetic dataset contains 100,000 sales lines and 15 fields. Its date range is 2025-01-01 to 2025-12-31. During package preparation, the file was profiled using pandas: it has 0 missing values across the loaded frame and 0 duplicate Transaction_ID values.

The sum of Total_Amount is INR 31,706,049.38. The average amount per line is INR 317.06; mean quantity per line is 2.02. The product category with the highest summed amount in this generated sample is Staples, at INR 5,797,173.59. The city label with the largest summed amount is Vijayawada, at INR 3,229,354.48.

These outputs are preliminary descriptive statistics computed from the exact bundled file. They should be reproduced by running `python/01_validate_data.py`, `python/02_eda_kpis.py`, and the SQL queries. When the student reruns the pipeline, update this chapter only from the actual outputs; avoid assuming the values will remain identical if the CSV is regenerated.

Because the generation logic chooses stores broadly evenly and product prices are illustrative, category and city rankings do not represent DMart’s real commercial performance. They serve as examples of how a data pipeline can answer a well-defined question.

Metric	Value from included file
Sales lines	100,000
Total sales amount	INR 31,706,049.38
Mean sales-line amount	INR 317.06
Mean quantity per line	2.02

VALIDATION, TESTING, AND RECONCILIATION

The validation plan checks the raw structure, null values, uniqueness, numeric ranges, formula integrity, and final totals. For this packaged dataset, the initial profile found no missing values and no duplicate sales-line keys, but these are observations of the current file rather than assumptions that should be hard-coded into software.

The most important cross-tool control is the sales total. Compare PostgreSQL `SUM(total_amount)`, pandas `df.Total_Amount.sum()`, Spark `SUM(Total_Amount)`, and the Power BI `[Sales Amount (INR)]` measure. Each value should match to two decimal places, provided all tools use the same records and no filters are applied.

The computed amount check verifies quantity multiplied by unit price and reduced by discount percentage. A mismatch can occur if a tool parses decimal values incorrectly, if percentages are interpreted as whole-number fractions incorrectly, or if rounding differs. Apply the same two-decimal rounding rule consistently.

Testing evidence should include terminal output, SQL result screenshots, notebook output, and Power BI totals. Record software versions and note any environment-specific workaround. An unexecuted script should be described as a template rather than a tested implementation.

Control	Expected result / action
Row count	100,000 source rows
Unique key	No duplicate Transaction_ID values
Null check	No missing required values
Formula check	Total_Amount agrees with calculated total
Cross-tool total	SQL = pandas = Spark = unfiltered Power BI
Evidence	Keep actual screenshots or logs from the run

CHAPTER 8: LIMITATIONS AND RECOMMENDATIONS

The largest limitation is data provenance: the transactions are simulated, not sourced from DMart. Product names, brands, prices, city assignments, payment methods, and customer IDs are illustrative. The project cannot support claims about actual demand, branch performance, customer behaviour, price competitiveness, market share, or company profitability.

The schema has no shared invoice ID, cost field, inventory on-hand balance, supplier history, or returns information. As a result, it cannot calculate accurate invoice-level average bill value, basket affinity, gross margin, stockout rates, supplier lead time, or return rates. Future work should add these fields only where a clear data definition exists.

A 100,000-row CSV is manageable for local analysis. It is not a convincing large-scale benchmark by itself. To test scalability, generate controlled datasets at 100,000, 1 million, and 5 million records, use comparable queries, and report observed run times with machine and software details.

Recommendations are to execute and test every pipeline stage, document any schema changes, use environment variables for credentials, avoid committing secrets, version code with Git, and present the final dashboard with an explicit synthetic-data disclaimer.

CONCLUSION AND FUTURE SCOPE

This project outlines an end-to-end retail analytics workflow, connecting relational data management with Python exploration, Spark transformations, Delta Lake storage, and business intelligence reporting. The repository is designed to make each stage visible and repeatable through scripts, documentation, a data dictionary, validation checks, and a notebook template.

The included data supports learning and descriptive analysis. Preliminary checks confirm the file is structurally suitable for the workflow, and calculated totals provide a benchmark for reconciling results across tools. The significance of the project lies in consistency and transparency: data fields are defined, calculations are testable, and claims are limited to what the dataset actually supports.

Future work can introduce a shared invoice ID and multiple lines per basket; add cost and inventory snapshots; build a date dimension; automate scheduled ingestion; benchmark larger row counts; apply incremental Delta updates; and deploy a Power BI report connected to a controlled data source. If authorized real data becomes available, it should be governed, anonymized where needed, and clearly separated from the synthetic learning file.

The report is a project draft until the student executes the scripts, replaces institutional placeholders, confirms outputs in the target environment, and inserts real screenshots of the finished workflow.

REFERENCES AND IMPLEMENTATION APPENDIX

[1] DMart / Avenue Supermarts Limited. Investor Relations - Financial Results. https://www.dmartindia.com/investor-relations/financial-results (accessed 28 October 2026). Used for general company context only.

[2] PostgreSQL Global Development Group. PostgreSQL Documentation: COPY. https://www.postgresql.org/docs/current/sql-copy.html (accessed 28 October 2026). Used for guidance on data import from files.

[3] Apache Software Foundation. Spark SQL. https://spark.apache.org/sql/ (accessed 28 October 2026). Used for structured processing context.

[4] Delta Lake. Documentation. https://docs.delta.io/ (accessed 28 October 2026). Used for Delta tables, schema validation, transactions, and version-history context.

[5] Microsoft. Power BI Documentation. https://learn.microsoft.com/en-us/power-bi/ (accessed 28 October 2026). Used for dashboard development context.

Implementation checklist: (a) run SQL database creation and import; (b) validate 100,000 rows; (c) capture category and monthly SQL results; (d) run Python scripts; (e) execute the Spark/Delta workflow in a compatible environment; (f) create and save the Power BI report; (g) replace placeholders and insert actual screenshots before academic submission.